System Design — Day 2

Topic: Databases — SQL vs NoSQL, Indexing, Replication

1. SQL vs NoSQL

The deciding factor isn't 'how often' data is used — it's the structure and scale of the data.

	SQL (relational)	NoSQL
Best for	Structured data, relationships, complex queries/joins	Massive scale, simple access patterns, flexible schema
Consistency	Strong (ACID)	Often eventual consistency, trades consistency for speed/scale
Example	User profiles, orders, payments	Likes, activity feeds, chat messages, sensor data

Example: user profiles (structured, relationships, moderate volume) → SQL. Likes on photos (billions of simple rows, massive write volume) → NoSQL.

2. Database Indexing

An index is a pre-built lookup structure (like a textbook's index) that lets the database jump straight to matching rows instead of scanning every row in the table (a 'full table scan').
Rule of thumb: index columns you frequently search, filter, or sort by (username, email, timestamps) — not every column.

3. Indexing Tradeoffs

- Slower writes — every INSERT/UPDATE must also update the index structure, not just the table.
- Extra disk space — an index is a separate data structure stored alongside the table.

Example of a bad index: the free-text 'bio' field — rarely searched/filtered on, and expensive to index due to length.

4. Replication

Keeping copies of a database on multiple servers to avoid a single point of failure.
Primary (master): handles all writes (INSERT/UPDATE/DELETE).
Replicas (slaves): copies that stay in sync with the primary and handle reads (SELECT).

Why split this way: redundancy (a replica can be promoted if the primary dies) and read scaling (most apps read far more than they write).

5. Replication Lag Bug (Read-After-Write Inconsistency)

Replication sync isn't instant — there's a small delay between the primary and its replicas.
Scenario: a user posts a comment (written to the primary) → the page immediately reloads to show it → that read gets routed to a replica → the replica hasn't received the new comment yet → the comment appears to be missing for a second or two, even though it posted successfully.
Real-world consequence: the user may think it failed and resubmit, creating a duplicate.
Common fix: "read-your-writes" consistency — route a user's own reads back to the primary right after their own write.

6. Key Takeaway

Choosing SQL vs NoSQL, deciding what to index, and understanding replication lag are core trade-off decisions that come up in nearly every system design interview — the goal is reasoning about the trade-off, not memorizing a 'correct' database choice.